GÉNIE INFORMATIQUE

COMPTE RENDU

TP 4 : CLOUD COMPUTING & DÉPLOIEMENT IA

L'API Iris et une base de données

Intégration de Redis et persistance des données avec Docker Compose

Réalisé par : MOTTAKI Hafsa

Filière : Génie Informatique – Data & AI

Encadré par : Pr. Mohammed AMEKSA

Année universitaire 2026–2027

Table des matières

1 Introduction et objectifs **1**
1.1 Architecture retenue 1

2 Mise en place de Redis dans l'API **2**
2.1 Ajout de la dépendance Python 2
2.2 Initialisation du client Redis 2
2.3 Compteur et endpoint de statistiques 3

3 Orchestration avec Docker Compose **4**
3.1 Configuration des services et des volumes 4
3.2 Validation du fichier Compose 4
3.3 Démarrage et contrôle des conteneurs 4

4 Tests fonctionnels **6**
4.1 Prédictions Random Forest et XGBoost 6

5 Persistance des données Redis **7**
5.1 Test avec docker compose down 7
5.2 Test avec docker compose down -v 7
5.3 Comparaison des résultats 8

6 Conclusion **9**

Table des figures

2.1 Ajout de la dépendance Redis dans le fichier requirements.txt. 2
2.2 Initialisation de la connexion Redis dans le fichier src/api.py. 3
2.3 Incrémentation du compteur et création de la route /stats. 3

3.1 Configuration du service Redis, des dépendances et des volumes dans compose.yaml. 4
3.2 Résultat de la commande docker compose config –services. 4
3.3 État des trois conteneurs : API et Redis healthy, entraînement terminé. 5

4.1 Prédictions réussies avec Random Forest et XGBoost et compteur égal à 2. . . 6

5.1 Conservation du compteur à 2 après down puis up. 7
5.2 Suppression et recréation des volumes ; compteur Redis réinitialisé à 0. 8

1. Introduction et objectifs

Ce travail pratique reprend l'API de classification Iris réalisée lors du TP précédent. Cette API, développée avec **FastAPI**, utilise deux modèles de machine learning : **Random Forest** et **XGBoost**. L'objectif du TP 4 est de lui ajouter une mémoire grâce à **Redis**, afin de compter le nombre de prédictions effectuées.

Le travail consiste à connecter l'API à Redis, à orchestrer les services avec Docker Compose, puis à comparer les effets de `docker compose down` et `docker compose down -v` sur la conservation des données.

1.1 Architecture retenue

Trois services sont définis dans le projet existant :

— **train** : entraîne les modèles et les sauvegarde dans le volume `iris-models` ;

— **api** : expose les routes de prédiction sur le port `8000` et utilise les modèles sauvegardés ;

— **base** : exécute Redis, qui conserve le compteur `total_predictions` dans le volume `redis-data`.

La communication entre l'API et Redis s'effectue sur le réseau interne Docker Compose, via le nom de service `base` et le port `6379`.

2. Mise en place de Redis dans l'API

2.1 Ajout de la dépendance Python

La bibliothèque `redis` a été ajoutée à `requirements.txt`, en complément des bibliothèques déjà utilisées pour les modèles et FastAPI.

```
redis>=5.0,<7.0
```

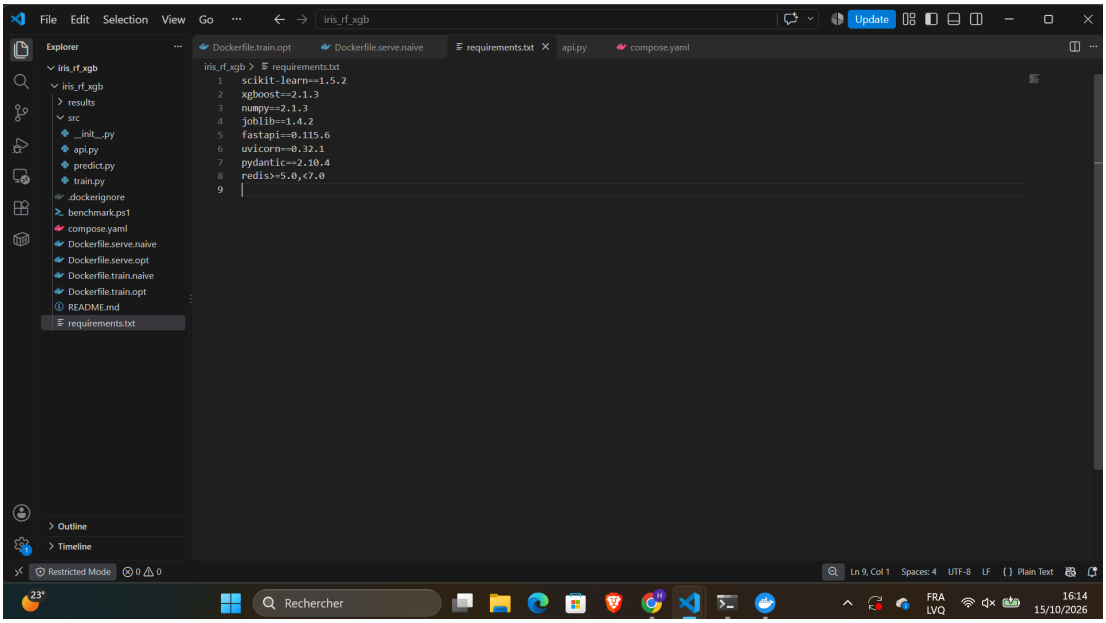


Figure 2.1 – Ajout de la dépendance Redis dans le fichier requirements.txt.

2.2 Initialisation du client Redis

Dans `src/api.py`, le client Redis lit les paramètres de connexion dans les variables d'environnement `REDIS_HOST` et `REDIS_PORT`. Par défaut, il se connecte au service `base` sur le port 6379.

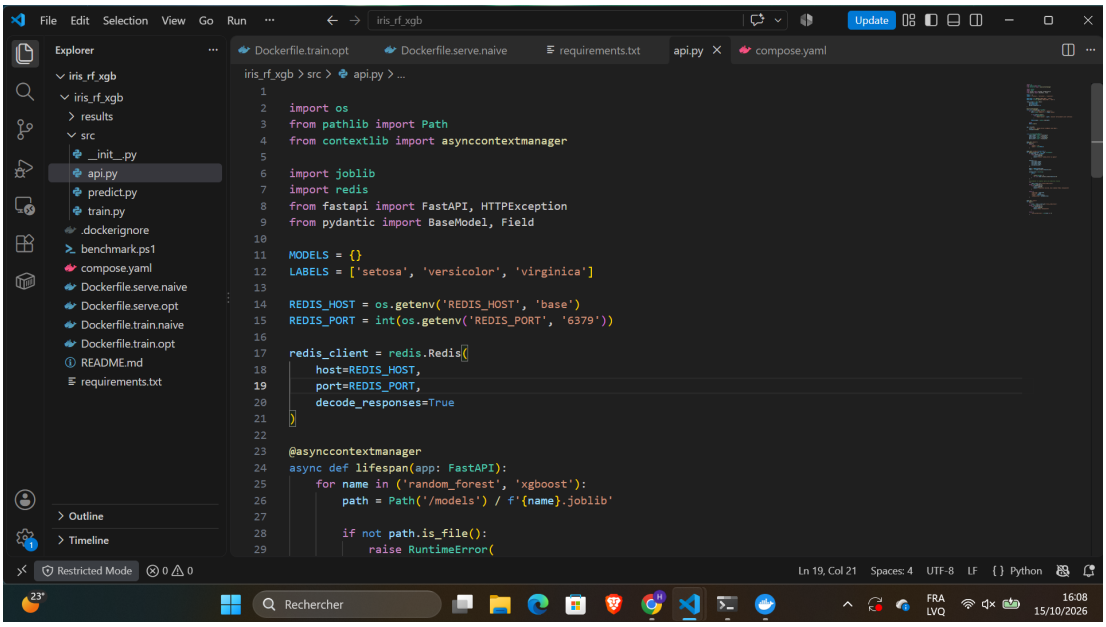


Figure 2.2 – Initialisation de la connexion Redis dans le fichier src/api.py.

2.3 Compteur et endpoint de statistiques

Après une prédiction réussie, l'instruction `redis_client.incr('total_predictions')` incrémente le compteur de 1. L'endpoint `GET /stats` lit ce compteur avec `redis_client.get(...)` et retourne le total. Le code prévoit également une réponse HTTP 503 si Redis est indisponible.

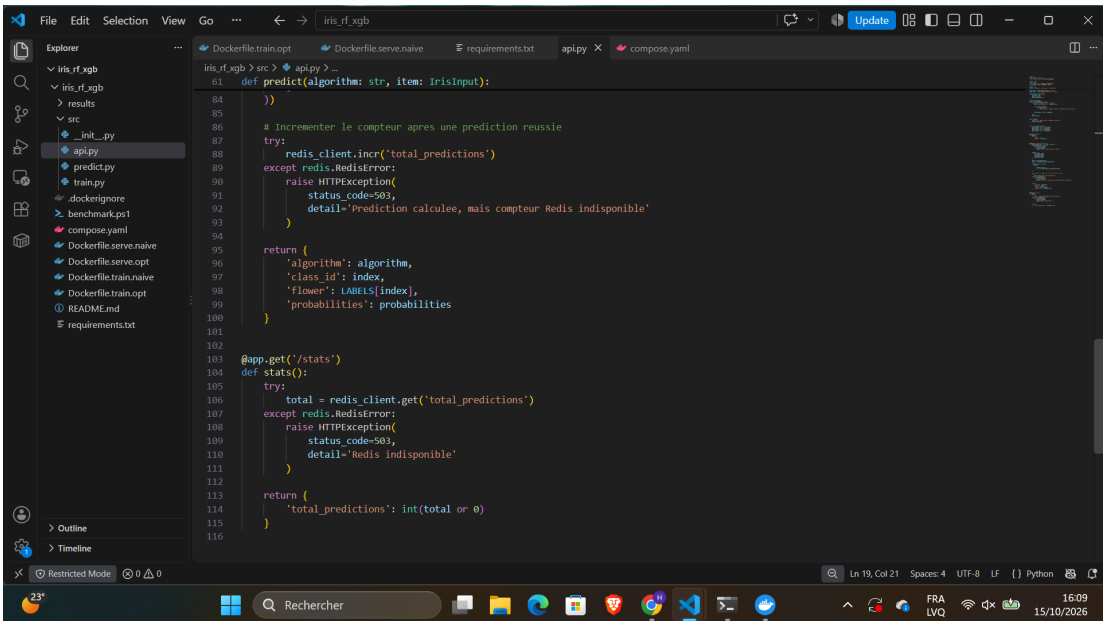


Figure 2.3 – Incrémentation du compteur et création de la route /stats.

3. Orchestration avec Docker Compose

3.1 Configuration des services et des volumes

Le fichier `compose.yaml` contient les services `train`, `api` et `base`. Redis utilise l'image `redis:7-alpine` et la commande `redis-server -appendonly yes` pour activer la persistance AOF. Le volume `redis-data:/data` conserve ses données. L'API attend que Redis soit en bonne santé et que l'entraînement se termine avec succès.

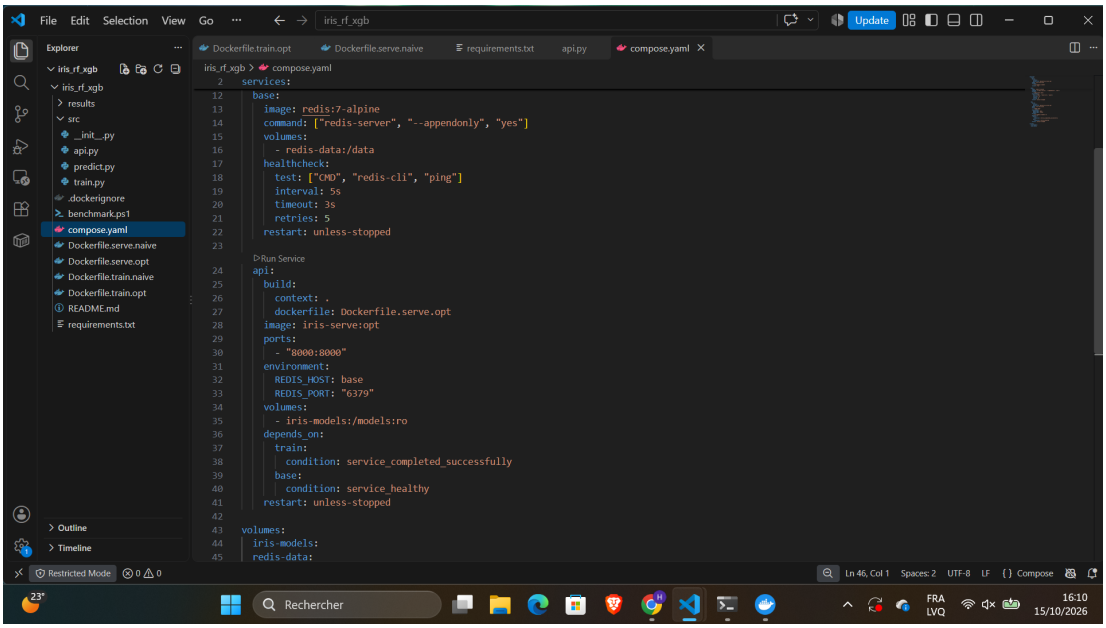


Figure 3.1 – Configuration du service Redis, des dépendances et des volumes dans compose.yaml.

3.2 Validation du fichier Compose

La syntaxe et la liste des services ont été vérifiées avec :

```
docker compose config
docker compose config --services
```

Le résultat confirme la présence des trois services `train`, `base` et `api`.

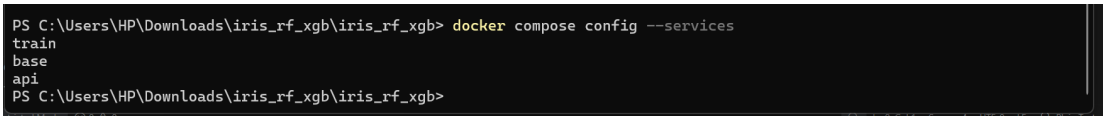


Figure 3.2 – Résultat de la commande docker compose config –services.

3.3 Démarrage et contrôle des conteneurs

Le projet a été lancé avec Docker Compose, puis les conteneurs ont été vérifiés :

```
docker compose up --build -d
docker compose ps -a
```

Les services `api` et `base` apparaissent comme `healthy`. Le service `train` est en état `Exited (0)`, ce qui est normal puisque son exécution se termine après l'entraînement.

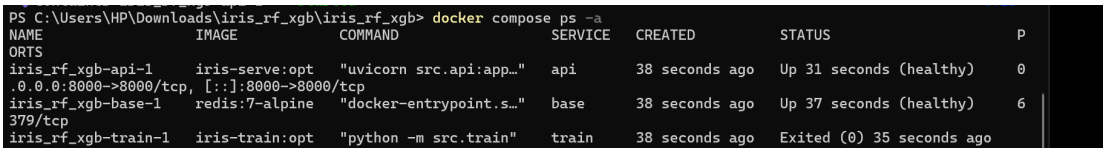


Figure 3.3 – État des trois conteneurs : API et Redis healthy, entraînement terminé.

4. Tests fonctionnels

4.1 Prédictions Random Forest et XGBoost

Une fleur Iris a été envoyée à l'API avec les caractéristiques suivantes : longueur du sépale 5.1, largeur du sépale 3.5, longueur du pétale 1.4 et largeur du pétale 0.2. Les deux modèles ont renvoyé la classe **setosa**.

```
POST /predict/random_forest
POST /predict/xgboost
GET /stats
```

Après ces deux prédictions, l'endpoint `/stats` a renvoyé `total_predictions = 2`, confirmant l'enregistrement du compteur dans Redis.



Figure 4.1 – Prédictions réussies avec Random Forest et XGBoost et compteur égal à 2.

5. Persistance des données Redis

5.1 Test avec docker compose down

Les conteneurs ont été arrêtés et supprimés, puis redémarrés sans supprimer les volumes :

```
docker compose down
docker compose up -d
Invoke-RestMethod -Uri "http://localhost:8000/stats" -Method Get
```

Après redémarrage, le compteur est resté à **2**. Ce résultat montre que les données sont conservées dans le volume Redis, même lorsque les conteneurs sont recréés.

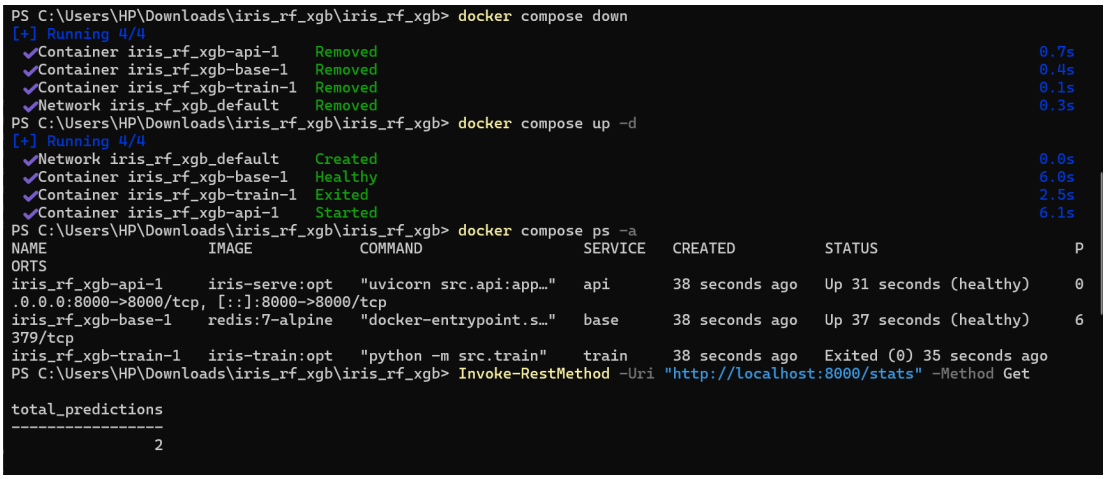


Figure 5.1 – Conservation du compteur à 2 après down puis up.

5.2 Test avec docker compose down -v

Le second test a utilisé l'option `-v`, qui supprime les volumes nommés du projet :

```
docker compose down -v
docker compose up -d
Invoke-RestMethod -Uri "http://localhost:8000/stats" -Method Get
```

Les volumes `redis-data` et `iris-models` ont été supprimés puis recréés au redémarrage. Le premier appel à `/stats` a échoué temporairement, l'API n'étant pas encore prête ; le second appel a réussi et a affiché **0**. Les modèles ont été régénérés par le service d'entraînement.

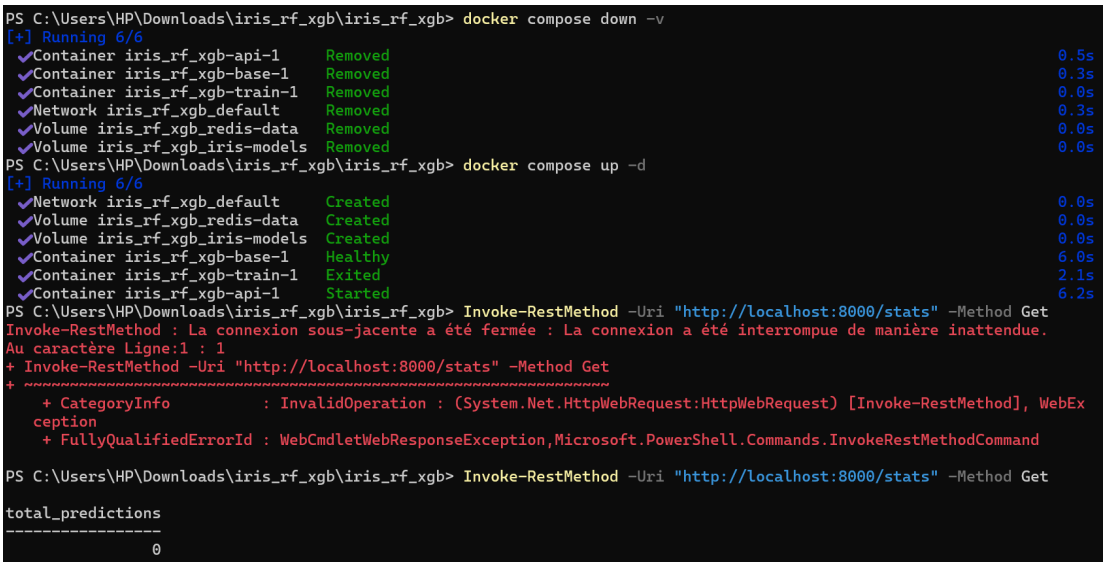


Figure 5.2 – Suppression et recréation des volumes ; compteur Redis réinitialisé à 0.

5.3 Comparaison des résultats

Opération	État du volume Redis	Compteur
Après deux prédictions	Présent	2
`down` puis `up`	Conservé	2
`down -v` puis `up`	Supprimé puis recréé	0

6. Conclusion

Ce TP a permis d'ajouter une mémoire à l'API Iris grâce à Redis et de faire fonctionner l'ensemble dans Docker Compose. Les prédictions Random Forest et XGBoost ont été réalisées avec succès et le compteur a correctement enregistré les deux requêtes.

Les tests de persistance ont montré que `docker compose down` conserve les volumes et donc le compteur, tandis que `docker compose down -v` supprime les volumes et réinitialise les données. Ces notions serviront lors du déploiement du couple API + Redis sur Kubernetes.